

SC-3346 — Maintenance (Derived) Pricing + COLA

Full investigation, root cause, fix, and live end-to-end validation for renewal-maintenance lines committing the wrong COLA price. Product in scope: Powertech Identity & Access Manager (BoKS).

VERDICT

FIXED — deployed and end-to-end proven on live data. A maintenance renewal now commits the correct COLA net **\$67.38 ($= 62.48 \times 1.0785$) instead of the frozen **\$60.64 / \$0** — the native engine applies the COLA uplift once the owned line renews as **Renew**.**

\$67.38

NOW COMMITS
(WAS \$60.64/\$0)

4/4

UNIT TESTS ·
100% CLASS COV

1 / 0

LINES TOTAL /
STRAY RRM

\$67.38

QUOTE ROLLUP
(NOT STALE)

6

DEAD-ENDS
RULED OUT

0

CATALOG / MODEL CHANGES

1 · THE DEFECT — IN BUSINESS TERMS

When a customer renews their BoKS maintenance, the renewal line priced at the **wrong amount** — a stale **\$60.64** (a one-period double-discounted fossil, $= 67.38 \times 0.90$) or **\$0** — instead of the correct cost-of-living-adjusted renewal of **\$67.38** (last year's net $\$62.48 \times 7.85\%$ COLA). The COLA arithmetic was never wrong; the value simply **never committed** to the line. Several prior remediation attempts failed because every one of them tried to *write the price* at reprice time — the wrong layer entirely.

2 · ROOT CAUSE — A SELLING-MODEL MISMATCH (FOUND VIA END-TO-END DEBUG)

- 1 New-business maintenance (PIA-PIA-RNM-PIAMBK) is sold under a **One-Time** selling model, so the asset the customer owns is One-Time — **not a renewable subscription**.
- 2 The platform's `initiateRenewal` therefore tags that asset's renewal as `QuoteAction.Type = "No Change"`, **Quantity 0** — which never enters the `Renew`-gated COLA path (`COLAUpliftoRenewalNet`). Term-Based subscriptions (beSECURE) renew as `Renew` and price correctly.
- 3 The legacy Year-2 AutoAdd rule then **deletes** that owned-asset line and substitutes an asset-less **Renewal-Maintenance** line (PIA-PIA-RRM-PIAM) with **no contributor** — the native `DerivedProductsRenewals` committer skips it, so its price freezes at the fossil or \$0.
- 4 **The fix:** stop discarding the owned line — **flip it to `Renew`** and let the native engine apply the COLA uplift to last year's net. No new pricing logic, no catalog change, no selling-model change.

3 · THE INVESTIGATION — PHASE BY PHASE

P1 Symptom triage & scope

Renewal-maintenance committing \$60.64 / \$0 vs the COLA-correct \$67.38. COLA math verified correct; the value was not persisting to the line.

P2 Two mechanisms separated

A1 (partner double-discount, posthook-version dependent) vs A2 (no priced node at all) — sequential, not competing. The earlier qty-based framing was tested and set aside.

P3 Contributor model established

A derived renewal prices only from a contributor: the prior owned Asset surfaced via `QuoteAction(Type=Renew).SourceAsset`, matched by product. No contributor → the committer skips the line.

P4 Reprice-time writes refuted (×3)

Direct `NetUnitPrice` write (read-only on insert), `PartnerNetPricePosthook`, and a procedure committer all returned success yet no-oped on a settled derived node.

P5 Self-healing candidate rejected

The "lever-d" procedure-commit change could self-heal existing fossils, but **could not be proven to fire** (the \$67.38 in those logs was prehook-sourced, byte-identical across runs) → rejected as unproven/unsafe.

P6 Born-net & PBEDP dead-ends

Hand-stamping a Renew QuoteAction *after* insert was too late (line settles at \$0). Mapping the owned product as a price contributor (PBEDP) is platform-blocked — it is itself a derived product.

P7

Decisive E2E debug — root cause exposed

Running the real `initiateRenewal` with debug logging showed it emitting **No Change / qty 0** for the One-Time maintenance asset — the line that never enters the COLA path. The fix writes itself: flip to `Renew`.

4 · WHY EARLIER FIXES COULD NOT WORK — THE DEAD-END LEDGER

ATTEMPTED LEVER	WHY IT CANNOT WORK
Write <code>NetUnitPrice</code> directly	DEAD engine-owned, read-only on insert
<code>PartnerNetPricePosthook</code> (reprice-time)	DEAD no-ops on a settled derived node
Procedure committer / RMPS ("lever-d")	DEAD returns success but could not be proven to fire; value was prehook-sourced
Map owned product as contributor (PBEDP)	DEAD platform-blocked — it is a derived product
Hand-attach a <code>Renew</code> <code>QuoteAction</code> after insert	DEAD too late; line settles un-priced and locks at \$0
Re-model the catalog (merge RNM/RRM)	UNNEEDED far higher blast radius; the flip achieves the same outcome

The unifying insight: every dead-end tried to *write the price*. None supplied the one thing the engine needs — a `Renew` line with a contributor. The fix does exactly that and lets the native engine compute and commit.

5 · THE FIX — DESIGN, CODE & WIRING

Apex · `RenewalMaintenanceFlip`

Selects renewal-quote lines that are owned-maintenance carryover stuck on `No Change`, and flips them:

- `QuoteAction.Type` → `Renew`
- line `StartQuantity = 0`, `Quantity = SourceAsset.Quantity`
- bulk-safe, idempotent, null/empty-safe
- `@InvocableMethod` for clean Flow binding

DEPLOYED API 64 · **4/4 tests pass** · **100% class coverage**

Flow • Fortra_Create_Renewal_Quote

Wired so every renewal automatically runs **flip** → **reprice**:

```
initiate renewal → populate dates →  
Flip Maintenance (apex) → Reprice (subflow) → $67.38 committed
```

Reprice = Place Sales Transaction, `pricingPref:Force` , `configurationPref:{configurationMethod:Skip}` .

6 • END-TO-END VALIDATION (FRESH LIVE RUN, 2026-06-14)

STAGE	OWNED MAINTENANCE LINE • PIA-PIA-RNM-PIAMBK
<code>initiateRenewal</code> → before flip	<code>No Change</code> qty 0 · StartQty 1 · Net = null · QA = No Change ← root cause reproduced live
after <code>flipToRenew</code>	<code>Renew</code> qty 1 · StartQty 0 · Net still null
after Force/Skip reprice	<code>Renew</code> qty 1 · Net = 67.38 · NetTotal 67.38 · Uplift 7.85% · QA = Renew

Survival — all-lines proof

An unfiltered query of **every** line on the renewal quote returned **exactly 1 line, 0 RRM lines**. The flipped owned line is the sole, correctly-priced maintenance line — no asset-less substitute survived.

Rollup — header not stale

Quote header totals all reconcile to the line: `Subtotal` = `TotalPrice` = `GrandTotal` = `ALE` = 67.38, `LineItemCount` 1. The original business symptom (a stale total) cannot re-surface.

Native-engine commit confirmed. The custom `COLACalculatedPrice__c` field read **0** on the settled line while `NetUnitPrice` = 67.38 — proving the *platform* renewal uplift committed the value once the line became a `Renew` node, not a custom Apex write. Downstream (Workday line-type, billing dates, ARR) is independent of the maintenance-type split. Test data was deleted; the canary asset remains intact at \$62.48.

7 · HONEST CAVEATS & SCOPE

Forward-only fix

The flip corrects renewals **as they are generated** (the wired flow / on-demand `flipToRenew`). It does **not** retroactively heal already-settled fossil lines on pre-existing Draft quotes — those need a separate remediation reprice.

Known residuals to confirm

- The exact platform reason a direct `NetUnitPrice` write no-ops is empirically strong (reproduced 3×) but not mechanistically pinned.
- The 7.85% rate comes from a COLA-rate custom metadata rule; the exact record was not re-pinned this pass — verify before prod.

8 · BEFORE PRODUCTION PROMOTION

Two verify-before-prod items carry forward: **re-pin the 7.85% BoKS COLA rate** to its `Maintenance_Rate__mdt` record (a known orphan-“platinum” rule exists elsewhere in this catalog), and **resolve the pre-existing `COLAUpliftTest.buildOverrideMap` coverage drift** (a separate test-suite issue that gates any procedure-side validate). A single production renewal smoke-test then confirms the flip + reprice commits **\$67.38** end to end.

FortraUAT (00DWC000006eUFF2A2) · canary asset 02iWC000008DmS1YAK · tests 4/4 + 100% class coverage · E2E all-lines + header-rollup verified · no Orders activated, no Workday events, Accepted quote 00781068 untouched · test data cleaned, canary intact · companion detail in the full MD